Constructors and Principles of OOP

CCS1200
Introduction to Object-Orientation

Dr Ioanna Stamatopoulou

Lecture Outline

- Role of constructors
- Default constructor
- Multiple constructors
- Use of constructors
- Garbage Collection (Destruction!)
- OOP Principles

Constructors

- A **Constructor** is a **special method** that is **responsible for two things**:
  - Creating objects:
    - Allocating memory
  - Initialising the attributes of the object:
    - Giving initial values to the attributes

Constructors (cont'd)

- More particularly:
  - When an attribute is of a primitive data type, the constructor provides an initial value
  - When an attribute is an instance of a class, the constructor creates/assigns an object to the attribute

Constructors: General Info

- The access modifier for a constructor is **public**
  - Anyone who wants to construct an object of this class should be allowed to do so
- The name of the constructor is the **exactly the same as the name of the class**
  - e.g. for the Time class, the default constructor is Time()
- A constructor **does not return a value** (we do **not even declare it as void)**
  - This is the main difference between a constructor and any other typical method

Types of Constructors

- Default constructor:
  - Provided by Java in case we do not define constructor(s) ourselves
  - Accepts no parameters
  - Always assigns the same default values to all attributes
- Non-Default Constructor(s)
  - Developer-defined
  - May have parameters so that we can define the initial values of the attributes

Default constructor

- The default constructor is automatically provided by Java if the developer does not define constructor(s) within a class
- It has no parameters
- Because there are no parameters, all the attributes are always initialised to default values
  - This means that if we create 2, 3, 4… objects with the default constructor, they will all have the same default values for their attributes

Default constructor: default values

- Default values for primitive data types:
  - 0 for numerical attributes (byte, short, int, long)
  - false for boolean attributes
  - the empty character for char
  - etc.
- Objects (non-primitive data types) are:
  - EITHER created using their default constructor to initialise every attribute
  - OR set to null

The Time class: Default Constructor

```
public class Time {
    //DEFAULT CONSTRUCTOR
    //initialises all attributes to zero
    Time() {
        setTime(0, 0, 0);
    }

    //alternative definition of default constructor
    /*
    Time() {
        hour = 0;
        minutes = 0;
        seconds = 0;
    */
    . . .
} //end of class
```

Non-default (developer-defined) constructors

- It is possible to define our own constructors
- Developer-defined constructors **may or may not have parameters**
  - Parameter values are used to **initialise the attributes** so it is possible to create many objects with different initial values every time
- You can define as many constructors as needed as long as they have a **different signature** (i.e. different parameter list)
  - Since they all have the same name, the parameter list is what differentiates the signature of the method

The Time class: Multiple developer-defined Constructors

```
public class Time {
   //accepts 3 parameters one for each attribute
   Time(int h, int m, int s) {
      setTime(h, m, s);
   }
   //accepts 2 parameters, for the hour and minutes
   Time(int h, int m) {
      setHour(h);
      setMinutes(m);
      seconds = 0;
   }
   //accepts 1 parameter, only for the hour
   Time(int h) {
      setHour(h);
      minutes = 0;
      seconds = 0;
   }
   . . .
}//end of class
```

Use of constructors

- The constructor is called using the **`new`** keyword according to the following syntax:

```
//creation of an object
Time arrivalTime = new Time() ;
```

class name (type)

object name

becomes a new

Time():
This is a call to the class's constructor

- Any other constructor can be used in the same way as long as we provide the necessary arguments for its parameters when we call it

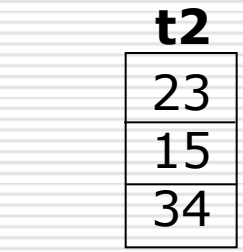
Use of the Time class Constructors

```
public class SomeOtherClass {
    . . .
    //Creation of an object with the Constructor
    //that does not accept arguments
    Time t1 = new Time();

    //Creation of an object with the Constructor
    //that accepts 3 arguments
    Time t2 = new Time(23,15,34);

    //Creation of an object with the Constructor
    //that accepts 2 arguments
    Time t3 = new Time(12,20);

    //Creation of an object with the Constructor
    //that accepts 1 argument
    Time t4 = new Time(22);
    . . .
} // end of class
```



The HotelRoom class

- Create as many constructors as you can for the HotelRoom class and call them to create objects

```
public class HotelRoom{
    // attributes (describe state)
    // they form the private (hidden) part
    private int number;
    private int numberOfBeds;
    private boolean available;

    …
} //end of class definition
```

Garbage Collection

- We saw how to construct objects but what about their destruction?
- No reason to worry about unused objects that may be using the computer memory (RAM) because:
- when an object stays unreferenced, thus not able to be used again, Java's automatic **garbage collector** is destroying it

OOP Principles

1st Principle of OOP

- **All objects are instances of a class**
- The behaviour of an object, i.e. the method used by an object in response to a message, is determined by its class
- All objects of a class use the same method (behave in a similar fashion) in response to similar messages

2nd Principle of OOP

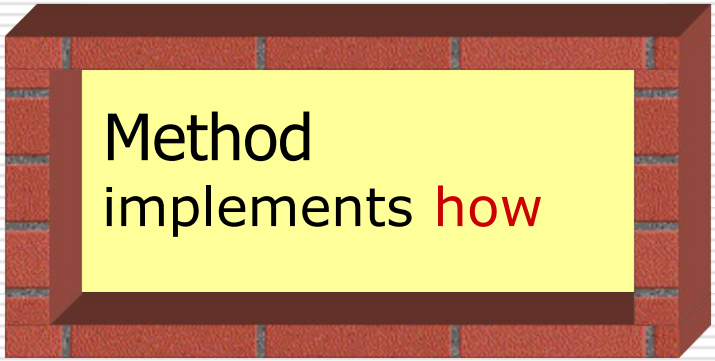
- Actions are initiated through **message passing** to an object
  - Message passing involves calling a **method** of the object
  - Message encodes a request and is accompanied by additional information (**arguments**) needed to carry out the request
  - The receiver accepts the message, which means it accepts the responsibility to carry out the request

Information Hiding

- Whoever calls a method does not need to know **how** the method is implemented
  - I do not care how the request will be satisfied as long as it is satisfied!
- My request describes **what** I want to achieve
- The method (**how**) is **hidden** from the caller

caller

describes what
my request is



object

Encapsulation

- Encapsulation links together state & behaviour
- Example: The hotel room
  - **STATE of being NOT AVAILABLE is linked to the METHOD of CHECKING IN**
  - Room cannot be unavailable without someone checking in
- State (attribute values) cannot be separated from behaviour (methods)
  - only the behaviour can change the state of an object

Encapsulation & Information hiding

- Encapsulation
  - hides private data of an object behind a public interface
  - protects object's data from external influences
    - object's state can only be changed by its methods of behaviour
  - hides implementation details
    - private part may change
    - public interface stays the same
- Information hiding
  - limited external access to internal data and operations